

PCB Defect Detection

Codebase & Architecture Review

Repository: AI-Builders-Iran/PCB-Defect-Detection | **Review date:** August 24, 2026

This report reviews the codebase as of the most recent commits ("Switch model format from ONNX to native PyTorch (.pt)" and "add new demo"), covering the shared inference pipeline, the FastAPI service, the Streamlit dashboard, and the Docker-based deployment configuration.

1. Executive Summary

PCB Defect Detection is a two-stage computer-vision system that first segments the PCB board within a frame (Model 1, a YOLO segmentation model) and then runs a dedicated defect-detection model (Model 2) only on the cropped board region, with ByteTrack used to track defects across video frames. The design choice to crop before running the second model is a sound one: it reduces the pixel area the second model has to process on every frame, and the code includes a full-frame fallback when no board is detected, so defects are not silently missed when segmentation fails.

The codebase is organized cleanly around a single shared pipeline module (`src/pipeline.py`) consumed by two independent front ends — a FastAPI REST service and a Streamlit dashboard — and ships with a working Docker/Docker Compose setup, a health-check endpoint, and bilingual (Persian/English) UI support. Overall the engineering quality is solid for a project at this stage: sensible defaults, defensive error handling around uploads and model loading, and clear separation of concerns.

The review did surface eight findings worth addressing before a production rollout, the most important being a stale ONNX-vs-PyTorch path mismatch between the `Dockerfile/.env.example` and the rest of the codebase (which would break a fresh Docker deployment), and the complete absence of authentication on the API. Both are addressed in detail in Section 5.

2. System Architecture

The core of the system is the two-stage inference pipeline implemented in `PCBPipeline` (`src/pipeline.py`), which both the API and the dashboard call into directly rather than duplicating detection logic:

- Stage 1 — Board segmentation: Model 1 (`best-pcb.pt`) runs on the full input frame and produces a segmentation mask for the PCB board. Multiple detected mask regions are merged with a bitwise OR into a single combined mask.
- Crop: a bounding rectangle is computed from the combined mask (with a 5-pixel margin) and the frame is cropped to that region before the second model runs — this is the main source of the claimed speed advantage over running defect detection on the full frame.

- Stage 2 — Defect detection: Model 2 (best_detect2.pt) runs on the cropped region (or the full frame if no board was found) with confidence/IOU thresholds from PipelineConfig, and optionally through YOLO's built-in `.track()` call with a ByteTrack configuration (bytetrack.yaml) for cross-frame identity persistence.
- Result assembly: detections are converted into DefectTrack records (class, confidence, bounding box, center, track id) and wrapped in a DetectionResult, which both front ends serialize to JSON via `to_dict()` and render visually via the shared `_draw_results()`/`_draw_fancy_box()` drawing routines.

Two independent consumers sit on top of this shared pipeline:

- FastAPI service (app/api/API_app.py) — a stateless REST API with synchronous image endpoints and a job-queue pattern for video (submit → poll status → download), backed by a ThreadPoolExecutor so long video jobs never block the async event loop.
- Streamlit dashboard (app/streamlit_app/app.py) — a self-contained UI that imports `src.pipeline` directly (it does not call the API), supports single-image and full-video workflows, and offers a bilingual EN/FA interface driven by a small in-file I18N dictionary.

Both services, plus the shared `src/` package, are packaged into a single Docker image with the front end selected at container-run time via the CMD (uvicorn for the API, streamlit run for the dashboard), and can be run together with `docker compose up`.

3. Component Review

3.1 Core Pipeline — `src/pipeline.py`

PCBPipeline is a well-structured, dependency-injected class: configuration is a single PipelineConfig dataclass (or an equivalent dict), and the class exposes three clear entry points — `process_image()` for single frames, `process()` for video/webcam streams with a progress callback, and the internal `_draw_results()` used by both front ends to render annotated output. Detection results are represented with typed dataclasses (DetectionResult, DefectTrack) rather than loose dictionaries, which keeps the JSON contract between the pipeline and its two consumers consistent.

Error handling around model loading is explicit: failures to load either YOLO model raise a RuntimeError with a clear message rather than failing silently, which the API layer turns into a 503 on /health. Per-detection extraction is wrapped in a try/except so a single malformed box does not abort processing of the rest of the frame.

Two implementation details are worth flagging for future maintenance (see Section 5 for full detail): the fallback logic that synthesizes a track id (`frame_num * 100 + i`) when the tracker does not supply one, and the fact that the pipeline reloads both models from disk every time a new PCBPipeline is constructed — which happens once per video job in the API layer.

3.2 REST API — `app/api/API_app.py`

The API is built on FastAPI with a lifespan handler that loads both models once at startup and stores the result in a module-level PipelineHolder; if loading fails, the service still starts (rather than crashing) and every `/predict/*` route reports a descriptive 503 via `ensure_models_ready()`, which is a good operability choice for container orchestration. The endpoint surface is small and consistent:

Method	Path	Purpose
GET	/health	Reports model-load status; used for container health checks.
POST	/predict/image	Synchronous single-image inference, returns JSON.
POST	/predict/image/annotated	Synchronous single-image inference, returns the annotated image binary.
POST	/predict/video	Starts an asynchronous video job in a thread pool, returns a <code>job_id</code> .
GET	/predict/video/{job_id}/status	Polls job progress and status.
GET	/predict/video/{job_id}/download/video	Downloads the annotated output video.
GET	/predict/video/{job_id}/download/json	Downloads the full per-frame JSON result.
DELETE	/predict/video/{job_id}	Deletes a job's output files and frees disk space.

Upload handling is streamed in 1 MB chunks with a running size check against `MAX_UPLOAD_MB` rather than buffering the whole file in memory first, and temporary files are consistently cleaned up in finally blocks. Video processing follows a standard async job pattern: `POST /predict/video` returns a `job_id` immediately, the actual work runs in a `ThreadPoolExecutor` (`run_video_job`), and the client polls `/predict/video/{job_id}/status` for progress and downloads outputs once status is done. Job bookkeeping is an in-memory dict (`_video_jobs`), which the code comments correctly identify as adequate for a demo/MVP but not for multi-worker or multi-replica deployments.

3.3 Streamlit Dashboard — `app/streamlit_app/app.py`

The dashboard imports `src.pipeline` directly rather than going through the API, and proactively fixes the classic `ModuleNotFoundError: No module named 'src'` problem by inserting the project root into `sys.path` at startup based on the script's own file location — a detail the README's troubleshooting table also calls out, so the fix and the documentation are consistent with each other.

Functionality is split into an Image tab and a Video tab, both driving the same `PCBPipeline`. The pipeline instance used by the Image tab is cached with `@st.cache_resource` so repeated single-image analyses do not reload the models on every rerun. The Video tab constructs a fresh `PipelineConfig/PCBPipeline` per run (mirroring the API's per-job pattern), streams progress into a `st.progress` bar via the same `progress_callback` mechanism used by `process()`, and re-encodes the output to H.264/yuv420p with `imageio-ffmpeg` when available, with a graceful fallback (and an explanatory caption) if `ffmpeg` isn't present. Sidebar controls (model paths, confidence/IOU thresholds, input resolution, tracking toggle, device) map directly onto `PipelineConfig` fields, and a simple `os.path.exists()` check on both model paths gates the run buttons and surfaces a clear "models not ready" warning. The bilingual EN/FA

support is implemented as a flat dictionary lookup rather than a full i18n framework, which is appropriate for the current scope but will not scale gracefully to a third language.

3.4 Deployment & Configuration

The Dockerfile builds from python:3.11-slim, installs the native libraries OpenCV/Ultralytics/ffmpeg need (libgl1, libglib2.0-0, libsm6, libxext6, ffmpeg), copies dependency manifests separately from application code for better layer caching, and — notably — runs the container as a non-root appuser and defines a HEALTHCHECK against /health. Model weights are deliberately excluded from the image (via .dockerignore) and mounted at runtime as read-only volumes, which keeps the image free of large/proprietary binaries and matches the instructions in the README. docker-compose.yml cleanly expresses the two-service topology (api and dashboard) sharing one built image, with the dashboard depending on the API for startup ordering even though the two do not currently communicate with each other at runtime.

The one inconsistency found in this layer is described in Section 5: the Dockerfile's baked-in ENV defaults and .env.example both still reference the old best-pcb.onnx / best_detect2.onnx filenames, while the application code and README have already migrated to best-pcb.pt / best_detect2.pt.

4. Engineering Practices — Strengths

- Single source of truth for inference logic: both the API and the dashboard call the same PCBpipeline, so detection behavior cannot drift between the two front ends.
- Sensible fail-open startup: model-load failures don't crash the API process; they surface as a 503 with the underlying error message, which is far easier to diagnose in a container environment than a crash-loop.
- Full-frame fallback when board segmentation finds nothing, so a Stage-1 miss degrades gracefully instead of dropping the frame's defects entirely.
- Async-safe long-running work: video processing is explicitly offloaded to a ThreadPoolExecutor so it cannot block the FastAPI event loop, and the same progress-callback abstraction is reused cleanly by the Streamlit dashboard.
- Careful resource handling: chunked upload reads with an enforced size cap, and temp files/directories are cleaned up in try/finally blocks across both the API and the dashboard.
- Container hardening basics are in place: non-root user, a real HEALTHCHECK, and model weights kept out of the image via .dockerignore rather than baked in.
- Typed data contracts: dataclasses for pipeline results and Pydantic models for every API response make the JSON shape predictable and self-documenting via /docs.

- Practical UX touches: an in-browser codec fallback message when ffmpeg re-encoding isn't available, cached model loading in Streamlit, and a bilingual (EN/FA) interface.

5. Findings & Recommendations

The table below lists concrete issues identified while cross-referencing the pipeline, API, dashboard, and deployment files against each other. Severity reflects likely production impact, not code elegance.

Finding	Severity	Description	Recommendation
ONNX / PyTorch path mismatch	High	Dockerfile ENV defaults and .env.example still point to best-pcb.onnx / best_detect2.onnx, while src/pipeline.py, API_app.py defaults and README were updated to the .pt filenames. A fresh Docker deployment using the documented defaults will fail to load models and /health will report "degraded".	Update Dockerfile ENV and .env.example to best-pcb.pt / best_detect2.pt, or keep both formats supported explicitly.
No authentication / open CORS	High	All FastAPI endpoints are unauthenticated and CORS_ORIGINS defaults to "*". Anyone who can reach the service can submit inference jobs, consume compute, and download any job's output by guessing/enumerating job_id.	Add an API key / OAuth layer before any non-local deployment; restrict CORS_ORIGINS; consider making job_id unguessable is not sufficient on its own.
In-memory video job state	Medium	_video_jobs is a plain process-local dict. Acknowledged in the code comments as fine for a demo, but it means job state is lost on restart and cannot be shared across multiple Uvicorn workers or replicas.	Move job metadata to Redis or a database before scaling beyond a single worker process.
Pipeline reloaded per video job	Medium	run_video_job() constructs a brand-new PCBPipeline (and therefore reloads both YOLO models from disk) for every submitted video, instead of reusing the already-loaded holder.pipeline instance used by the image endpoints.	Reuse a shared, thread-safe pipeline instance (or a small pool) for video jobs to cut latency and memory churn.
Track-ID fallback collision risk	Low	When the tracker does not return an id, _extract_defects() falls back to frame_num * 100 + i. With more than 100 defects in a single frame this scheme can collide with the next frame's ids.	Use a monotonically increasing global counter or a UUID for the fallback id instead of a frame-derived formula.
Feature parity gap: max_frames	Low	The Streamlit dashboard exposes a "max frames to process" control, but the equivalent PipelineConfig.max_frames option is not exposed as a query parameter on POST /predict/video.	Add an optional max_frames query parameter to the video endpoint for parity and easier smoke-testing.
No automated tests in the repository	Medium	The project structure (per README) does not include a tests/ directory; there is no visible unit/integration test coverage for the pipeline, API, or job lifecycle.	Add pytest-based tests, at minimum for _extract_defects(), the crop/fallback logic in _process_frame(), and the FastAPI endpoints (via httpx / TestClient with mocked models).

Finding	Severity	Description	Recommendation
Output video codec on API downloads	Low	Videos processed via the API are written with the mp4v FourCC (cv2.VideoWriter). The Streamlit dashboard re-encodes to H.264 for browser playback, but the API's /download/video endpoint serves the raw mp4v file as-is.	If the annotated video is ever played back inline in a browser (rather than downloaded), apply the same ffmpeg H.264 re-encode step used in the dashboard.

6. Suggested Next Steps

- Fix the ONNX/PyTorch path mismatch first — it is a one-line change in two files but will silently break any fresh Docker deployment that relies on the documented defaults.
- Add a minimal auth layer (API key header at minimum) before exposing the API outside a trusted network, and tighten CORS_ORIGINS accordingly.
- Introduce a pytest suite covering the pipeline's crop/fallback branch, the track-id fallback, and the API's job lifecycle (queued → processing → done/failed) with mocked YOLO models so tests run without GPU/model weights.
- Move video-job state out of process memory (Redis or a lightweight database) ahead of any horizontal scaling or multi-worker deployment.
- Reuse a single loaded pipeline instance for video jobs instead of constructing a new PCBPipeline (and reloading both models) per job.
- Expose max_frames on the video API endpoint for parity with the dashboard, and consider basic API versioning
- (e.g., /v1/predict/...) ahead of any breaking changes.

7. Conclusion

PCB Defect Detection is a well-structured, production-leaning project: the two-stage crop-then-detect architecture is a genuinely good performance/accuracy trade-off rather than an academic exercise, and the codebase reflects real attention to operability — health checks, graceful degradation, non-root containers, and consistent temp-file hygiene across both front ends. The issues raised in Section 5 are the normal gap between an MVP and a hardened production service (state management, auth, test coverage) plus one concrete migration bug (the ONNX/PyTorch path mismatch) that should be fixed immediately since it affects the documented "quick start" path. None of the findings point to a flawed architecture — they are incremental hardening steps on top of a solid foundation.